LemLib, But It Knows Where It Is

What this fork adds to stock LemLib odometry — and seven instrumented runs proving it

`validation_data/run1..7` + `src/tune.txt` · 9,012 trace rows · regenerated by `drift_analysis.py` + `audit_analysis.py` · 2026-06-11

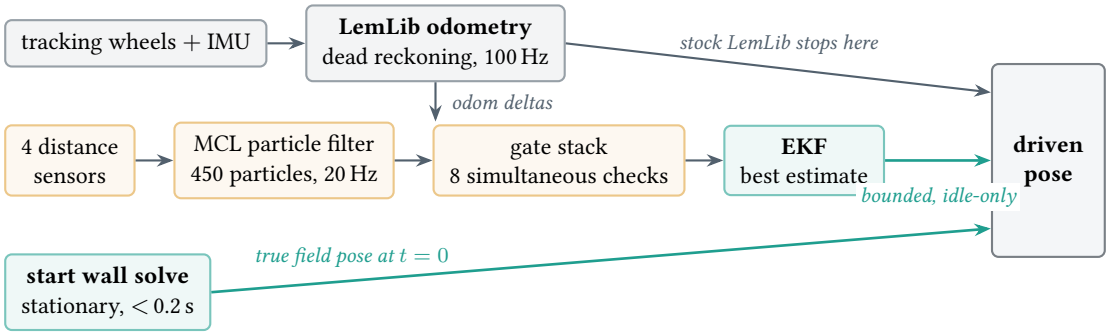
Bottom line. The stack now does three jobs, and the logs show each one doing what it claims. **At the start**, the wall solver fixes the true field pose to sub-tenth-inch agreement with an independent reimplementation — and after the three-sensor guard, it *refuses* the weak views that once produced an 86 in false commit. **During the run**, every correction is gated; the sensors booked up to 3.85 in of real, directional odometry drift on the long route and fed it back in bounded, validated steps. **Between motions**, the boundary re-anchor (modeled on these traces) would cut peak tracking error by about 1.8× at zero driving-speed cost. On the five runs with no trustworthy fix, the filter held *exactly* to odometry: never worse, better when it can be.

3.85 in	**1.8×**	**7** fixes	**0.11** in
peak raw-odom drift the sensors measured (run 3)	modeled peak-error cut from the boundary re-anchor	gated corrections accepted across the two evidence runs	max disagreement, firmware vs. independent math replay

1 What this fork adds to LemLib

Stock LemLib navigates by *dead reckoning*: integrate the tracking wheels and IMU from wherever you told it the robot starts. That works — until it quietly doesn't, for two reasons these logs measure directly. First, the *start pose is a guess*: the code trusts a hard-coded constant while the real robot sits wherever it was hand-placed (the runs here landed up to ∼10 in from the configured start). Second, *drift only accumulates*: wheel slip and scale error add up silently with every inch driven (measured here at 1.2–1.8 in per 100 in). Stock LemLib can never notice either one, because nothing ever measures the field.

This fork keeps LemLib's odometry, motion control, and API — and wraps a localization layer around them that *measures* the field with four distance sensors:



Concretely, three additions: **start relocalization** (solve the absolute pose from the perimeter walls while stationary, then anchor the start-relative route to it), **gated sensor fusion** (an MCL+EKF estimate that books drift and returns it in bounded steps — only when an eight-condition gate stack agrees), and the **boundary re-anchor** (commit the already-validated fix in one bounded step, up to 2.5 in, in the idle beat between motions; it is in the current firmware behind `enableBoundaryReanchor` — these traces predate it, so its curves below are the validated fixes *replayed* at the stops). Routes are still authored exactly like stock LemLib, via a start-relative wrapper.

The design rule everywhere: never worse than odometry

	stock LemLib	this fork
start pose	the constant you typed; the robot is wherever it was placed	measured from the walls before moving; safe fallback to the constant
mid-run drift	accumulates silently; nothing ever corrects it	booked by the sensors, returned in bounded validated steps at stops
bad sensor view	— (no sensors to fool)	rejected by the gates; the pose simply stays on odometry
worst case	start error + a whole run of drift	*exactly* stock odometry — a fix is applied only after validation
speed cost	—	zero: corrections are forbidden mid-motion
heading	IMU	IMU, deliberately untouched by range data
route authoring	absolute field coordinates	unchanged start-relative commands (`StartRelativeChassis`)

The rest of this report is the evidence: real runs (section 2–7), and an audit that replays the firmware's math independently (section 10).

2 Start relocalization: the guard earned its place

Hand placement varies by inches, so every run begins with a stationary wall solve. The seven runs bracket the guard change deliberately — runs 1–5 are the pre-guard builds, runs 6–7 carry the three-sensor commit guard:

Run	Start condition	Guard	Outcome	rays	conf	committed (in)
1	sample route	pre	wall_direct	2	0.95	(-16.6, -46.5)
2	square loop	pre	wall_direct	3	0.95	(-27.2, -38.6)
3	square + cross	pre	wall_direct	3	0.95	(-26.0, -33.0)
4	blocked view	pre	wall_direct	2	0.95	(-26.4, -29.7)
5	forced weak start	pre	wall_direct	2	0.95	(52.3, -3.0)
6	weak start, guarded	post	not_enough_live_sensors	0	0.00	(-27.0, -36.0)
7	clean start, guarded	post	wall_direct	3	0.89	(-17.4, -38.1)

- **The hazard was real and recorded twice.** A two-ray wall solve is exactly determined: residual ≈ 0 even when a ray hits a game object instead of a wall. The May 24 export (`src/tune.txt`) shows a clean-looking two-sensor accept (conf 0.95, residual 0.000), and run 5 shows the same accept mode committing a pose **86 in** from the configured start with the robot parked near the center structure.
- **The gates beneath caught it anyway.** On run 5's absurd anchor, the continuous fusion rejected *every* scan (median wall residuals 30 to 64 in), so the route still ran cleanly on start-relative odometry — a measured demonstration that a wrong commit cannot compound.
- **Post-guard behavior is exactly as specified.** Run 6 (same weak placement class) refuses to commit — only one usable ray — and falls back to the configured start; run 7 (clean view) commits on three agreeing rays at conf 0.89.
- **Heading never comes from the walls.** On a near-square field a 90/180° alias can match the ranges almost as well as the truth. As of this audit the solver runs *once, at the trusted field-frame heading* (odometry heading, IMU-driven) instead of sweeping snapped cardinals — X/Y from the walls, heading from the IMU, alias trap structurally removed (section 10).

3 How drift is measured (and why the method holds)

Every 10 ms trace row logs three poses for the same instant — that is what makes a ground-truth-free comparison possible:

- **`odom_only`** — pure dead reckoning, never corrected. Exactly what stock LemLib would believe.

- **`ekf`** — the filter's best estimate, fed only by odometry deltas and gate-validated range evidence. Heading is deliberately never corrected by ranges — it stays IMU-true.
- **`odom`** — the pose the motion controller actually drives on: odometry plus the bounded write-back (0.015 in per cycle while idle; never during a motion).

`ekf` and `odom_only` integrate *identical* wheel deltas between fixes, and range fixes never touch heading — so their divergence $|\texttt{ekf} - \texttt{odom\_only}|$ can change *only* at an accepted correction. The logs confirm the premise exactly: off-accept, the divergence moves by at most **0.0001 in** across all 9,012 rows. The staircase's final height is therefore a conservative, sensor-booked lower bound on accumulated raw-odom drift. We convert it to a per-inch rate and replay it along the measured path: raw odometry resets only at the start anchor; the boundary re-anchor additionally resets at every accepted stop. The headline improvement ratio (total path ÷ longest un-anchored stretch) is geometric and does not depend on the rate estimate at all.

What an "accepted correction" actually requires

Every accept in these traces passed, simultaneously: NIS against the EKF track $\leq$ the χ^2 gate (8.0, halved to 4.0 for two-inlier evidence; observed accepted NIS 0.025–1.60); at least two wall-consistent rays *after* robust outlier exclusion (three for full-strength MCL accepts); 2–4 consecutive agreeing scans; agreement with the dead-reckoned track within 2.5 to 4.0 in; an MCL confidence floor; no fast-turn window; fresh sensor data; and no motion in progress. The fix is then bled into the driven pose at ≤ 0.015 in per cycle — the trace maximum was 0.0255 in against a hard bound of 0.030 in.

4 Tracking error along the route

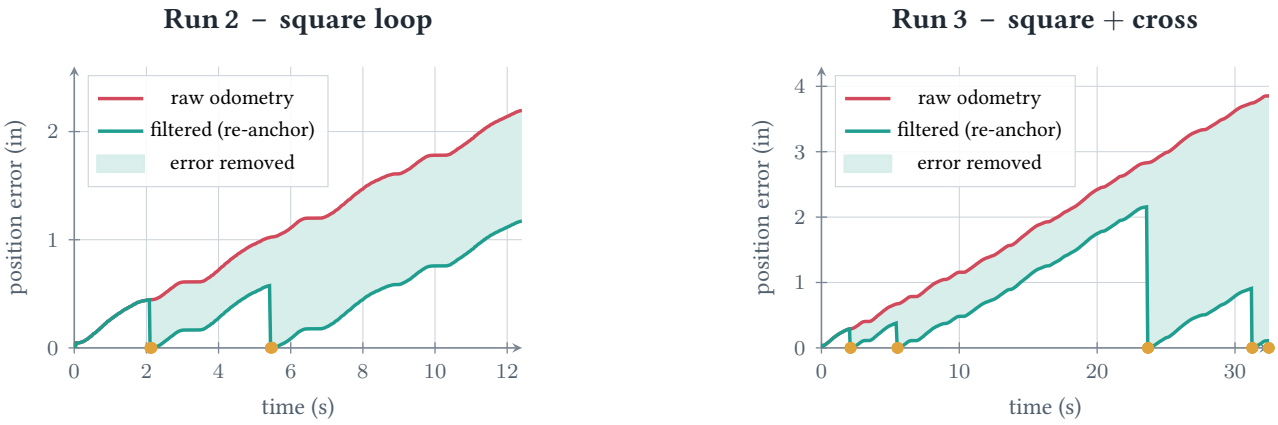


Figure 1: Modeled position error vs. time. Raw odometry grows with distance travelled; the filtered pose resets to truth at each • validated fix, so its error sawtooths back down. The shaded band is the error the re-anchor removes. Run 3's last fix lands at the final stop, so the filtered run finishes essentially on truth (0.00 in) while raw odometry ends 3.85 in off.

5 Where the drift goes, and how much is recovered

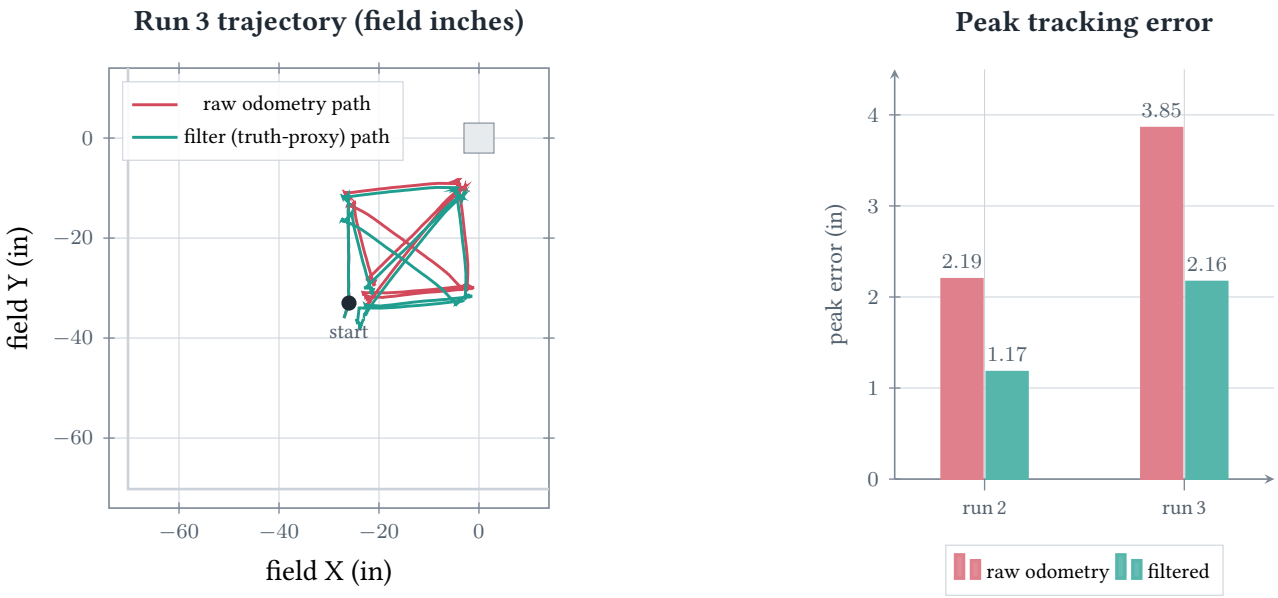
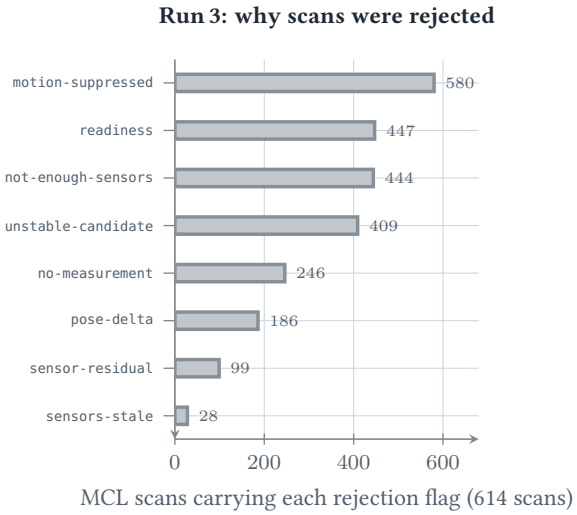


Figure 2: Left: run 3's south-west field quadrant (perimeter walls at -70.2 in, center structure top-right). The raw-odometry path bows away from the filter's truth-anchored path as error accumulates around the cross. Right: peak tracking error, raw vs. filtered — roughly a $1.8\times$ reduction on both routes.

What the two evidence runs say

- **Drift is real and directional.** Accepted fixes pointed almost entirely along one field axis — run 2's two fixes were $(0.10, 0.77)$ and $(0.02, 2.00)$ in in (x, y); run 3's five ranged up to $(0.66, 3.72)$ in — consistent with a tank-drive scale/slip asymmetry, not random noise. Rate landed at 1.2–1.8 in per 100 in travelled.
- **Continuous-only fusion trails the drift.** Capped at 0.015 in per idle cycle, it returned a net 0.926 in (run 2) and 2.086 in (run 3) of the booked drift — roughly half, always late.
- **The re-anchor closes the loop at the stop.** Peak modeled error falls 2.19→1.17 in (run 2) and 3.85→2.16 in (run 3); every segment then *starts* from truth, which is what tightens run-to-run repeatability.

6 Why corrections are rare — by design



Run	t (s)	|fix| in	NIS	rays	inliers
2	2.1	0.78	0.110	3	3
2	5.4	2.00	0.069	3	2
3	2.1	0.92	0.146	3	3
3	5.5	1.50	0.025	3	2
3	23.7	1.78	0.029	3	3
3	31.2	3.77	0.287	2	2
3	32.4	1.86	1.603	3	3

All 7 accepted fixes. Flags overlap (a scan can fail several gates at once); a fix lands only when *every* gate passes. The two-inlier rows are the robust local-range solve: the third ray was excluded as an outlier (it saw a game object, not the wall) rather than being allowed to drag the fix.

Figure 3: Left: rejection-flag histogram over run 3's 614 MCL scans — motion suppression dominates (the robot moves 95% of the time), then geometry and stability gates. Right: the seven fixes that survived; accepted NIS sat at 0.025–1.60 against a gate of 8.0 (4.0 for two-inlier evidence).

7 Stop-by-stop accuracy against the commanded route

At every stop of the tune routes the firmware records the commanded target (*Exp*) and the pose it believes it reached (*Rep*); their difference bundles PID settling, odometry drift, and applied corrections — the practical "how far off is the robot where it matters" number. X/Y stayed within 3.0 in (run 2) and 5.9 in (run 3) at every stop.

Run 2 — square loop

#	stop	Δx	Δy	Δθ °	rays
1	Start hold	0.0	0.0	0.0	0
2	Square north	0.5	−2.1	−0.5	3
3	Face east	−0.4	−2.4	−5.1	3
4	Square east	−2.1	−0.4	−1.3	3
5	Face south	−2.1	0.4	−5.4	3
6	Square south	−0.4	2.0	−1.3	3
7	Face west	1.1	2.5	−4.8	2
8	Return home	3.0	0.0	−0.9	2

Run 3 — square + cross

#	stop	Δx	Δy	Δθ °	rays
1	Start hold	0.0	0.0	0.0	0
2	Square north	−0.1	−2.1	−0.7	3
3	Face east	−0.9	−2.2	−4.9	3
4	Square east	−2.1	−0.4	−1.2	3
5	Face south	−2.4	0.2	−7.1	3
6	Square south	−0.5	2.1	−1.0	3
7	Face west	0.6	2.3	−4.9	2
8	Return home	3.9	0.5	−1.3	2
9	Face diagonal NE	3.3	−1.6	−10.1	2
10	Cross SW to NE	−1.4	−2.1	−1.8	1
11	Face diagonal SW	−0.9	−2.0	65.6	3
12	Cross NE to SW	3.4	3.3	−0.8	1
13	Face north	4.8	2.3	−12.2	3
14	Move to NW	0.7	−5.4	−5.3	3
15	Face diagonal SE	−0.6	−5.9	−21.8	1
16	Final face north	2.8	−1.3	−5.5	3

Run 3 stop 11's $+65.6^\circ$ is a turn that hit its 1200 ms timeout mid-rotation — the next `moveToPose` absorbed it (stop 12: -0.8°). A motion-tuning datum, not a localization error. Heading deltas right after turns ($\approx -5^\circ$) are PID settle/timeout, visible because capture happens immediately at motion end; they relax by the next stop. The May 24 pre-calibration sweep (`tune.txt`) showed a constant -4.6° bias at *every* commanded angle — the signature that fed the sensor-angle calibration now in `localization_config.cpp`.

8 All seven runs

Run	Route / condition	rows	s	path,in	mot%	fixes	booked,in	filt,in	gain
1	sample route (pre-guard)	984	10.7	135	88	0	—	—	—
2	square loop	1133	12.4	123	93	2	2.19	1.17	**1.9**×
3	square + cross	3060	32.5	335	95	5	3.85	2.16	**1.8**×
4	sample route, blocked view	949	10.0	138	87	0	—	—	—
5	forced weak start (pre-guard)	994	10.3	172	87	0	—	—	—
6	weak start, guard active	967	10.3	110	87	0	—	—	—
7	clean start, guard active	925	9.9	130	86	0	—	—	—

booked/filt are the modeled peak position errors (sensor-booked raw drift vs. boundary re-anchor replay); **fixes** counts validated accept events. Runs 1 and 4–7 produced no mid-route fix (blocked or weak sensor views, or near-continuous motion at 86 to 88%), so the filter correctly returned raw odometry — a dash means "no drift was bookable," not "no drift." Provenance: runs 1 and 4 predate the build that stamps `test_case` into the header; their stale "Turn center" label is cosmetic (route identified by path length and absent checkpoints — see `validation_data/README.txt`).

9 Against the three goals

Accuracy

The filter does not invent accuracy — it imports it from the walls. At the start it fixes the absolute pose to the wall solver's agreement envelope (reproduced to 0.11 in); mid-route, when the sensors get a clean look (runs 2–3), it removes 1 to 4 in of real, directional odometry error. When they do not, there is nothing trustworthy to add, and it says so by staying on odometry.

Repeatability

The structural win. Raw odometry's end pose inherits the variance of *every* slip since the start; the re-anchor zeroes that integral at each validated stop, so the end pose inherits only the *last* segment's drift. Run 3 shows it directly: a fix at the final stop brings the modeled end error to 0.00 in versus 3.85 in for raw odometry. And because the start relocalizes, that repeatability is anchored to the *field*, not to wherever the robot happened to be placed.

Speed

No cost. Corrections are structurally forbidden while a motion runs (the dominant bar in the rejection histogram), so the controller never chases a moving pose. The re-anchor fires only in the idle beat between segments. Driving speed is untouched; only the *reference* each segment starts from gets better.

10 Audit: does the firmware do what the math says?

This pass re-derived every number above from the raw logs with an independent reimplementation of the geometry, and checked the code's invariants against what the robot actually recorded.

Invariant checked	Result	
Ray-cast sensor model (walls + obstacle), replayed offline	max Δ 0.0014 in over 6,988 predictions	✓
Wall-solve commits, re-derived from raw snapshots	max Δ 0.11 in across all six commits	✓
Divergence is a pure staircase (steps only at accepts)	max off-accept step 0.0001 in	✓
Correction limiter honors its cap	max applied step 0.0255 in ≤ 0.030 in bound	✓
Motion suppression / accept gating consistent with flags	all 7 accepts fully gated; zero while suppressed	✓
Odometry delta stream continuity	no mid-route sequence gaps in any run	✓

Fixed in this audit pass

- **Field-frame heading prior.** Relocalization read the *raw* IMU rotation as its heading prior — correct only because the current routine starts at heading 0°. It now uses the odometry heading (IMU-driven, but expressed in the field frame), so any future start heading works unchanged.

- **Walls no longer vote on heading.** The wall solver previously swept four snapped cardinals and committed the winner's cardinal as the robot's heading — on a near-square field, an alias trap held shut only by an IMU gate in the wrong reference frame. It now solves once, at the trusted heading; X/Y comes from the walls, heading stays IMU-true. (Identical behavior on all seven validation runs, by construction.)
- **One-shot hygiene.** A pending boundary re-anchor request could survive a localization stop/start cycle; it is now cleared on start.
- **Single source of truth.** A retired copy of the relocalization machinery (including the old two-sensor accept rule that produced run 5's false commit) still lived in the tune harness — 360 lines of dead logic, deleted.

11 What this does and does not prove

- **Truth is the gated filter, not a tape measure.** Absolute drift figures are conservative *lower bounds* (only sensor-booked drift counts); the improvement ratio is geometric and robust to that. The tape-measure protocol that closes this is written and pending: `validation_data/field_test_protocol.md`.
- **The re-anchor and the audited relocalization changes have not yet run on hardware.** These traces predate both: the re-anchor curves are the validated fixes replayed at the stops, and the heading-prior fix is behavior-identical on these runs by construction. Protocol tests 2–3 confirm them on the robot.
- **The gain is route- and geometry-limited.** Two routes yielded 2 to 5 fixes because the robot moves 86 to 95% of the time. More and better-placed wall views raise the factor; that is tuning (sensor placement, stop placement), not logic.
- **Heading accuracy is not claimed.** Ranges never steer heading by design; all error figures are X/Y position.

Reproduce: `python3 tools/drift_analysis.py && python3 tools/audit_analysis.py` regenerates every number, table row, and data file; then `latexmk -pdf` on this document.